

第22章 DMA—直接存储区访问

本章参考资料：《STM32F4xx 中文参考手册》DMA 控制器章节。

学习本章时，配合《STM32F4xx 中文参考手册》DMA 控制器章节一起阅读，效果会更好，特别是涉及到寄存器说明的部分。本章内容专业名称较多，内容丰富也较难理解，但非常有必要细读研究。

特别说明，本章内容是以 STM32F4xxx 系列资源讲解。

22.1 DMA 简介

DMA(Direct Memory Access,直接存储区访问)为实现数据高速在外设寄存器与存储器之间或者存储器与存储器之间传输提供了高效的方法。之所以称之为高效，是因为 DMA 传输实现高速数据移动过程无需任何 CPU 操作控制。从硬件层次上来说，DMA 控制器是独立于 Cortex-M4 内核的，有点类似 GPIO、USART 外设一般，只是 DMA 的功能是可以快速移动内存数据。

STM32F4xx 系列的 DMA 功能齐全，工作模式众多，适合不同编程环境要求。STM32F4xx 系列的 DMA 支持外设到存储器传输、存储器到外设传输和存储器到存储器传输三种传输模式。这里的外设一般指外设的数据寄存器，比如 ADC、SPI、I2C、DCMI 等等外设的数据寄存器，存储器一般是指片内 SRAM、外部存储器、片内 Flash 等等。

外设到存储器传输就是把外设数据寄存器内容转移到指定的内存空间。比如进行 ADC 采集时我们可以利用 DMA 传输把 AD 转换数据转移到我们定义的存储区中，这样对于多通道采集、采样频率高、连续输出数据的 AD 采集是非常高效的处理方法。

存储区到外设传输就是把特定存储区内容转移至外设的数据寄存器中，这种多用于外设的发送通信。

存储器到存储器传输就是把一个指定的存储区内容拷贝到另一个存储区空间。功能类似于 C 语言内存拷贝函数 memcopy，利用 DMA 传输可以达到更高的传输效率，特别是 DMA 传输是不占用 CPU 的，可以节省很多 CPU 资源。

22.2 DMA 功能框图

STM32F4xx 系列的 DMA 可以实现外设寄存器与存储器之间或者存储器与存储器之间传输三种模式，这要得益于 DMA 控制器是采样 AHB 主总线的，可以控制 AHB 总线矩阵来启动 AHB 事务。图 22-1 为 DMA 控制器的框图。

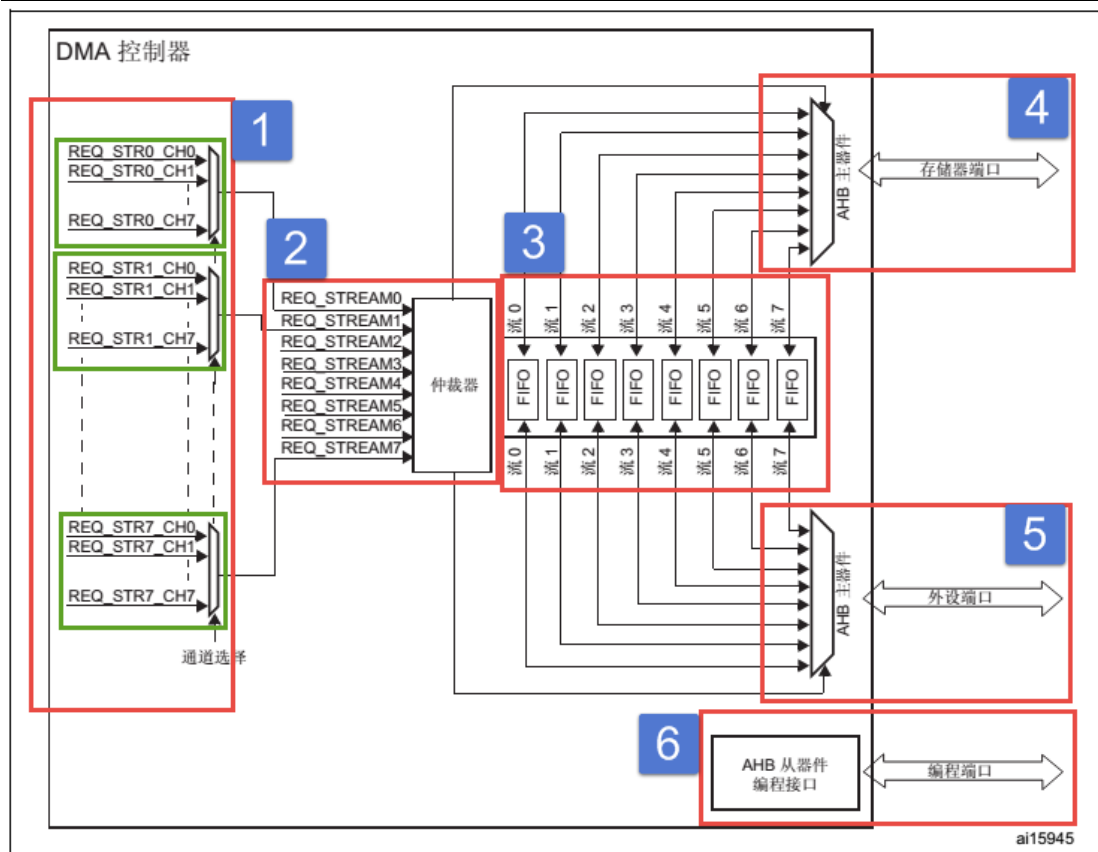


图 22-1 DMA 框图

1. ①外设通道选择

STM32F4xx 系列资源丰富，具有两个 DMA 控制器，同时外设繁多，为实现正常传输，DMA 需要通道选择控制。每个 DMA 控制器具有 8 个数据流，每个数据流对应 8 个外设请求。在实现 DMA 传输之前，DMA 控制器会通过 DMA 数据流 x 配置寄存器 DMA_SxCR(x 为 0~7，对应 8 个 DMA 数据流)的 CHSEL[2:0]位选择对应的通道作为该数据流的目标外设。

外设通道选择要解决的主要问题是决定哪一个外设作为该数据流的源地址或者目标地址。

DMA 请求映射情况参考表 22-1 和表 22-2。

表 22-1 DMA1 请求映射

外设请求	数据流 0	数据流 1	数据流 2	数据流 3	数据流 4	数据流 5	数据流 6	数据流 7
通道 0	SPI3_RX		SPI3_RX	SPI2_RX	SPI2_TX	SPI3_TX		SPI3_TX
通道 1	I2C1_RX		TIM7_UP		TIM7_UP	I2C1_RX	I2C1_TX	I2C1_TX
通道 2	TIM4_CH1		I2S3_EXT_RX	TIM4_CH2	I2S2_EXT_TX	I2S3_EXT_TX	TIM4_UP	TIM4_CH3

通道 3	I2S3_EXT_RX	TIM2_UP TIM2_CH3	I2C3_RX	I2S2_EXT_RX	I2C3_TX	TIM2_CH1	TIM2_CH2 TIM2_CH4	TIM2_UP TIM2_CH4
通道 4	UART5_RX	USART3_RX	UART4_RX	USART3_TX	UART4_TX	USART2_RX	USART2_TX	UART5_TX
通道 5	UART8_TX	UART7_TX	TIM3_CH4 TIM3_UP	UART7_RX	TIM3_CH1 TIM3_TRIG	TIM3_CH2	UART8_RX	TIM3_CH3
通道 6	TIM5_CH3 TIM5_UP	TIM5_CH4 TIM5_TRIG	TIM5_CH1	TIM5_CH4 TIM5_TRIG	TIM5_CH2		TIM5_UP	
通道 7		TIM6_UP	I2C2_RX	I2C2_RX	USART3_TX	DAC1	DAC2	I2C2_TX

表 22-2 DMA2 请求映射

外设请求	数据流 0	数据流 1	数据流 2	数据流 3	数据流 4	数据流 5	数据流 6	数据流 7
通道 0	ADC1		TIM8_CH1 TIM8_CH2 TIM8_CH3		ADC1		TIM1_CH1 TIM1_CH2 TIM1_CH3	
通道 1		DCMI	ADC2	ADC2		SPI6_TX	SPI6_RX	DCMI
通道 2	ADC3	ADC3		SPI5_TX	SPI5_TX	CRYPT_OUT	CRYPT_IN	HASH_IN
通道 3	SPI1_RX		SPI1_RX	SPI1_TX		SPI1_TX		
通道 4	SPI4_RX	SPI4_TX	USART1_RX	SDIO		USART1_RX	SDIO	USART1_TX
通道 5		USART6_RX	USART6_RX	SPI14_RX	SPI4_TX		USART6_TX	USART6_TX
通道 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_COM TIM1_TRIG	TIM1_UP	TIM1_CH3	
通道 7		TIM8_UP	TIM8_CH1	TIM8_CH2	TIM8_CH3	SPI5_RX	SPI5_TX	TIM8_CH4 TIM8_TRIG TIM8_COM

每个外设请求都占用一个数据流通道，相同外设请求可以占用不同数据流通道。比如 SPI3_RX 请求，即 SPI3 数据发送请求，占用 DMA1 的数据流 0 的通道 0，因此当我们使用该请求时，我们需要在把 DMA_S0CR 寄存器的 CHSEL[2:0] 设置为 “000”，此时相同数据流的其他通道不被选择，处于不可用状态，比如此时不能使用数据流 0 的通道 1 即 I2C1_RX 请求等等。

查阅表 22-1 可以发现 SPI3_RX 请求不仅仅在数据流 0 的通道 0，同时数据流 2 的通道 0 也是 SPI3_RX 请求，实际上其他外设基本上都有两个对应数据流通道，这两个数据流通道都是可选的，这样设计是尽可能提供多个数据流同时使用情况选择。

2. ②仲裁器

一个 DMA 控制器对应 8 个数据流，数据流包含要传输数据的源地址、目标地址、数据等信息。如果我们需要同时使用同一个 DMA 控制器(DMA1 或 DMA2)多个外设请求时，那必然需要同时使用多个数据流，那究竟哪一个数据流具有优先传输的权利呢？这就需要仲裁器来管理判断了。

仲裁器管理数据流方法分为两个阶段。第一阶段属于软件阶段，我们在配置数据流时可以通过寄存器设定它的优先级别，具体配置 DMA_SxCR 寄存器 PL[1:0]位，可以设置为非常高、高、中和低四个级别。第二阶段属于硬件阶段，如果两个或以上数据流软件设置优先级一样，则他们优先级取决于数据流编号，编号越低越具有优先权，比如数据流 2 优先级高于数据流 3。

3. ③FIFO

每个数据流都独立拥有四级 32 位 FIFO(先进先出存储器缓冲区)。DMA 传输具有 FIFO 模式和直接模式。

直接模式在每个外设请求都立即启动对存储器传输。在直接模式下，如果 DMA 配置为存储器到外设传输那 DMA 会见一个数据存放在 FIFO 内，如果外设启动 DMA 传输请求就可以马上将数据传输过去。

FIFO 用于在源数据传输到目标地址之前临时存放这些数据。可以通过 DMA 数据流 xFIFO 控制寄存器 DMA_SxFCR 的 FTH[1:0]位来控制 FIFO 的阈值，分别为 1/4、1/2、3/4 和满。如果数据存储量达到阈值级别时，FIFO 内容将传输到目标中。

FIFO 对于要求源地址和目标地址数据宽度不同时非常有用，比如源数据是源源不断的字节数据，而目标地址要求输出字宽度的数据，即在实现数据传输时同时把原来 4 个 8 位字节的数据拼凑成一个 32 位字数据。此时使用 FIFO 功能先把数据缓存起来，分别根据需要输出数据。

FIFO 另外一个作用使用于突发(burst)传输。

4. ④存储器端口、⑤外设端口

DMA 控制器实现双 AHB 主接口，更好利用总线矩阵和并行传输。DMA 控制器通过存储器端口和外设端口与存储器和外设进行数据传输，关系见图 22-2。DMA 控制器的功能是快速转移内存数据，需要一个连接至源数据地址的端口和一个连接至目标地址的端口。

DMA2(DMA 控制器 2)的存储器端口和外设端口都是连接到 AHB 总线矩阵，可以使用 AHB 总线矩阵功能。DMA2 存储器和外设端口可以访问相关的内存地址，包括有内部 Flash、内部 SRAM、AHB1 外设、AHB2 外设、APB2 外设和外部存储器空间。

到存储器	外设端口	存储器端口			突发	禁止	
			外设	禁止	单次	允许	禁止
存储器到外设	AHB 存储器端口	AHB 外设端口	DMA	允许	突发	禁止	允许
					单次	允许	
			外设	禁止	突发	禁止	禁止
					单次	允许	
存储器到存储器	AHB 存储器端口	AHB 存储器端口	仅 DMA	禁止	突发	禁止	禁止
					单次	禁止	禁止

1. DMA 传输模式

DMA2 支持全部三种传输模式，而 DMA1 只有外设到存储器和存储器到外设两种模式。模式选择可以通过 DMA_SxCR 寄存器的 DIR[1:0]位控制，进而将 DMA_SxCR 寄存器的 EN 位置 1 就可以使能 DMA 传输。

在 DMA_SxCR 寄存器的 PSIZE[1:0]和 MSIZE[1:0]位分别指定外设和存储器数据宽度大小，可以指定为字节(8 位)、半字(16 位)和字(32 位)，我们可以根据实际情况设置。直接模式要求外设和存储器数据宽度大小一样，实际上在这种模式下 DMA 数据流直接使用 PSIZE，MSIZE 不被使用。

2. 源地址和目标地址

DMA 数据流 x 外设地址 DMA_SxPAR(x 为 0~7)寄存器用来指定外设地址，它是一个 32 位数据有效寄存器。DMA 数据流 x 存储器 0 地址 DMA_SxM0AR(x 为 0~7)寄存器和 DMA 数据流 x 存储器 1 地址 DMA_SxM1AR(x 为 0~7)寄存器用来存放存储器地址，其中 DMA_SxM1AR 只用于双缓冲模式，DMA_SxM0AR 和 DMA_SxM1AR 都是 32 位数据有效的。

当选择外设到存储器模式时，即设置 DMA_SxCR 寄存器的 DIR[1:0] 位为“00”，DMA_SxPAR 寄存器为外设地址，也是传输的源地址，DMA_SxM0AR 寄存器为存储器地址，也是传输的目标地址。对于存储器到存储器传输模式，即设置 DIR[1:0] 位为“10”时，采用与外设到存储器模式相同配置。而对于存储器到外设，即设置 DIR[1:0]位为“01”时，DMA_SxM0AR 寄存器作为为源地址，DMA_SxPAR 寄存器作为目标地址。

3. 流控制器

流控制器主要涉及到一个控制 DMA 传输停止问题。DMA 传输在 DMA_SxCR 寄存器的 EN 位被置 1 后就进入准备传输状态，如果有外设请求 DMA 传输就可以进行数据传输。很多情况下，我们明确知道传输数据的数目，比如要传 1000 个或者 2000 个数据，这样我们就可以在传输之前设置 DMA_SxNDTR 寄存器为要传输数目值，DMA 控制器在传输完这么多数目数据后就可以控制 DMA 停止传输。

DMA 数据流 x 数据项数 DMA_SxNDTR(x 为 0~7)寄存器用来记录当前仍需要传输数目，它是一个 16 位数据有效寄存器，即最大值为 65535，这个值在程序设计是非常有用也是需

要注意的地方。我们在编程时一般都会明确指定一个传输数量，在完成一次数目传输后 DMA_SxNDTR 计数值就会自减，当达到零时就说明传输完成。

如果某些情况下在传输之前我们无法确定数据的数目，那 DMA 就无法自动控制传输停止了，此时需要外设通过硬件通信向 DMA 控制器发送停止传输信号。这里有一个大前提就是外设必须是可以发出这个停止传输信号，只有 SDIO 才有这个功能，其他外设不具备此功能。

4. 循环模式

循环模式相对应于一次模式。一次模式就是传输一次就停止传输，下一次传输需要手动控制，而循环模式在传输一次后会按照相同配置重新传输，周而复始直至被控制停止或传输发生错误。

通过 DMA_SxCR 寄存器的 CIRC 位可以使能循环模式。

5. 传输类型

DMA 传输类型有单次(Single)传输和突发(Burst)传输。突发传输就是用非常短时间结合非常高数据信号率传输数据，相对正常传输速度，突发传输就是在传输阶段把速度瞬间提高，实现高速传输，在数据传输完成后恢复正常速度，有点类似达到数据块“秒传”效果。为达到这个效果突发传输过程要占用 AHB 总线，保证要求每个数据项在传输过程不被分割，这样一次性把数据全部传输完才释放 AHB 总线；而单次传输时必须通过 AHB 的总线仲裁多次控制才传输完成。

单次和突发传输数据使用具体情况参考表 22-4。其中 PBURST[1:0]和 MBURST[1:0]位是位于 DMA_SxCR 寄存器中的，用于分别设置外设和存储器不同节拍数的突发传输，对应为单次传输、4 个节拍增量传输、8 个节拍增量传输和 16 个节拍增量传输。PINC 位和 MINC 位是寄存器 DMA_SxCR 寄存器的第 9 和第 10 位，如果位被置 1 则在每次数据传输后数据地址指针自动递增，其增量由 PSIZE 和 MSIZE 值决定，比如，设置 PSIZE 为半字大小，那么下一次传输地址将是前一次地址递增 2。

表 22-4 DMA 传输类型

AHB 主端口	项目	单次传输	突发传输
外设	寄存器	PBURST[1:0]=00, PINC 无要求	PBURST[1:0]不为 0, PINC 必须为 1
	描述	每次 DMA 请求就传输一次字节/半字/字(取决于 PSIZE)数据	每次 DMA 请求就传输 4/8/16 个(取决于 PBURST[1:0])字节/半字/字(取决于 PSIZE)数据
存储器	寄存器	MBURST[1:0]=00, MINC 无要求	MBURST[1:0]不为 0, MINC 必须为 1
	描述	每次 DMA 请求就传输一次字节/半字/字(取决于 MSIZE)数据	每次 DMA 请求就传输 4/8/16 个(取决于 MBURST[1:0])字节/半字/字(取决于 MSIZE)数据

突发传输与 FIFO 密切相关，突发传输需要结合 FIFO 使用，具体要求 FIFO 阈值一定要是内存突发传输数据量的整数倍。FIFO 阈值选择和存储器突发大小必须配合使用，具体参考表 22-5。

表 22-5 FIFO 阈值配置

MSIZE	FIFO 级别	MBURST=INCR4	MBURST=INCR8	MBURST=INCR16
字节	1/4	4 个节拍的 1 次突发	禁止	禁止
	1/2	4 个节拍的 2 次突发	8 个节拍的 1 次突发	
	3/4	4 个节拍的 3 次突发	禁止	
	满	4 个节拍的 4 次突发	8 个节拍的 2 次突发	
半字	1/4	禁止	禁止	禁止
	1/2	4 个节拍的 1 次突发		
	3/4	禁止		
	满	4 个节拍的 2 次突发		
字	1/4	禁止	禁止	禁止
	1/2			
	3/4			
	满	4 个节拍的 1 次突发		

6. 直接模式

默认情况下，DMA 工作在直接模式，不使能 FIFO 阈值级别。

直接模式在每个外设请求都立即启动对存储器传输的单次传输。直接模式要求源地址和目标地址的数据宽度必须一致，所以只有 PSIZE 控制，而 MSIZE 值被忽略。突发传输是基于 FIFO 的所以直接模式不被支持。另外直接模式不能用于存储器到存储器传输。

在直接模式下，如果 DMA 配置为存储器到外设传输那 DMA 会见一个数据存放在 FIFO 内，如果外设启动 DMA 传输请求就可以马上将数据传输过去。

7. 双缓冲模式

设置 DMA_SxCR 寄存器的 DBM 位为 1 可启动双缓冲传输模式，并自动激活循环模式。双缓冲不应用与存储器到存储器的传输。双缓冲模式下，两个存储器地址指针都有效，即 DMA_SxM1AR 寄存器将被激活使用。开始传输使用 DMA_SxM0AR 寄存器的地址指针所对应的存储区，当这个存储区数据传输完 DMA 控制器会自动切换至 DMA_SxM1AR 寄存器的地址指针所对应的另一块存储区，如果这一块也传输完成就再切换至 DMA_SxM0AR 寄存器的地址指针所对应的存储区，这样循环调用。

当其中一个存储区传输完成时都会把传输完成中断标志 TCIF 位置 1，如果我们使能了 DMA_SxCR 寄存器的传输完成中断，则可以产生中断信号，这个对我们编程非常有用。另外一个非常有用的信息是 DMA_SxCR 寄存器的 CT 位，当 DMA 控制器是在访问使用 DMA_SxM0AR 时 CT=0，此时 CPU 不能访问 DMA_SxM0AR，但可以向 DMA_SxM1AR 填充或者读取数据；当 DMA 控制器是在访问使用 DMA_SxM1AR 时 CT=1，此时 CPU 不能访问 DMA_SxM1AR，但可以向 DMA_SxM0AR 填充或者读取数据。另外在未使能 DMA 数据流传输时，可以直接写 CT 位，改变开始传输的目标存储区。

双缓冲模式应用在需要解码程序的地方是非常有效的。比如 MP3 格式音频解码播放，MP3 是被压缩的文件格式，我们需要特定的解码库程序来解码文件才能得到可以播放的 PCM 信号，解码需要一定的实际，按照常规方法是读取一段原始数据到缓冲区，然后对缓冲区内容进行解码，解码后才输出到音频播放电路，这种流程对 CPU 运算速度要求高，很容易出现播放不流畅现象。如果我们使用 DMA 双缓冲模式传输数据就可以非常好的解决这个问题，达到解码和输出音频数据到音频电路同步进行的效果。

8. DMA 中断

每个 DMA 数据流可以在发送以下事件时产生中断：

- 1) 达到半传输：DMA 数据传输达到一半时 HTIF 标志位被置 1，如果使能 HTIE 中断控制位将产生达到半传输中断；
- 2) 传输完成：DMA 数据传输完成时 TCIF 标志位被置 1，如果使能 TCIE 中断控制位将产生传输完成中断；
- 3) 传输错误：DMA 访问总线发生错误或者在双缓冲模式下试图访问“受限”存储器地址寄存器时 TEIF 标志位被置 1，如果使能 TEIE 中断控制位将产生传输错误中断；
- 4) FIFO 错误：发生 FIFO 下溢或者上溢时 FEIF 标志位被置 1，如果使能 FEIE 中断控制位将产生 FIFO 错误中断；
- 5) 直接模式错误：在外设到存储器的直接模式下，因为存储器总线没得到授权，使得先前数据没有完成被传输到存储器空间上，此时 DMEIF 标志位被置 1，如果使能 DMEIE 中断控制位将产生直接模式错误中断。

22.4 DMA 初始化结构体详解

标准库函数对每个外设都建立了一个初始化结构体 `xxx_InitTypeDef` (xxx 为外设名称)，结构体成员用于设置外设工作参数，并由标准库函数 `xxx_Init()` 调用这些设定参数进入设置外设相应的寄存器，达到配置外设工作环境的目的。

结构体 `xxx_InitTypeDef` 和库函数 `xxx_Init` 配合使用是标准库精髓所在，理解了结构体 `xxx_InitTypeDef` 每个成员意义基本上就可以对该外设运用自如了。结构体 `xxx_InitTypeDef` 定义在 `stm32f4xx_xxx.h` (后面 xxx 为外设名称) 文件中，库函数 `xxx_Init` 定义在 `stm32f4xx_xxx.c` 文件中，编程时我们可以结合这两个文件内注释使用。

DMA_InitTypeDef 初始化结构体

```
1 typedef struct {  
2     uint32_t DMA_Channel;           //通道选择  
3     uint32_t DMA_PeripheralBaseAddr; //外设地址  
4     uint32_t DMA_Memory0BaseAddr;   //存储器 0 地址  
5     uint32_t DMA_DIR;                //传输方向  
6     uint32_t DMA_BufferSize;        //数据数目
```

```
7    uint32_t DMA_PeripheralInc;      //外设递增
8    uint32_t DMA_MemoryInc;         //存储器递增
9    uint32_t DMA_PeripheralDataSize; //外设数据宽度
10   uint32_t DMA_MemoryDataSize;    //存储器数据宽度
11   uint32_t DMA_Mode;              //模式选择
12   uint32_t DMA_Priority;          //优先级
13   uint32_t DMA_FIFOMode;          //FIFO 模式
14   uint32_t DMA_FIFOThreshold;     //FIFO 阈值
15   uint32_t DMA_MemoryBurst;       //存储器突发传输
16   uint32_t DMA_PeripheralBurst;   //外设突发传输
17 } DMA_InitTypeDef;
```

- 5) **DMA_Channel**: DMA 请求通道选择, 可选通道 0 至通道 7, 每个外设对应固定的通道, 具体设置值需要查表 22-1 和表 22-2; 它设定 DMA_SxCR 寄存器的 CHSEL[2:0]位的值。例如, 我们使用模拟数字转换器 ADC3 规则采集 4 个输入通道的电压数据, 查表 22-2 可知使用通道 2。
- 6) **DMA_PeripheralBaseAddr**: 外设地址, 设定 DMA_SxPAR 寄存器的值; 一般设置为外设的数据寄存器地址, 如果是存储器到存储器模式则设置为其中一个存储区地址。ADC3 的数据寄存器 ADC_DR 地址为((uint32_t)ADC3+0x4C)。
- 7) **DMA_Memory0BaseAddr**: 存储器 0 地址, 设定 DMA_SxM0AR 寄存器值; 一般设置为我们自定义存储区的首地址。我们程序先自定义一个 16 位无符号整形数组 ADC_ConvertedValue[4]用来存放每个通道的 ADC 值, 所以把数组首地址(直接使用数组名即可)赋值给 DMA_Memory0BaseAddr。
- 8) **DMA_DIR**: 传输方向选择, 可选外设到存储器、存储器到外设以及存储器到存储器。它设定 DMA_SxCR 寄存器的 DIR[1:0]位的值。ADC 采集显然使用外设到存储器模式。
- 9) **DMA_BufferSize**: 设定待传输数据数目, 初始化设定 DMA_SxNDTR 寄存器的值。这里 ADC 是采集 4 个通道数据, 所以待传输数目也就是 4。
- 10) **DMA_PeripheralInc**: 如果配置为 DMA_PeripheralInc_Enable, 使能外设地址自动递增功能, 它设定 DMA_SxCR 寄存器的 PINC 位的值; 一般外设都是只有一个数据寄存器, 所以一般不会使能该位。ADC3 的数据寄存器地址是固定并且只有一个所以不能使能外设地址递增。
- 11) **DMA_MemoryInc**: 如果配置为 DMA_MemoryInc_Enable, 使能存储器地址自动递增功能, 它设定 DMA_SxCR 寄存器的 MINC 位的值; 我们自定义的存储区一般都是存放多个数据的, 所以使能存储器地址自动递增功能。我们之前已经定义了一个包含 4 个元素的数字用来存放数据, 使能存储区地址递增功能, 自动把每个通道数据存放到对应数组元素内。

- 12) DMA_PeripheralDataSize: 外设数据宽度, 可选字节(8 位)、半字(16 位)和字(32 位), 它设定 DMA_SxCR 寄存器的 PSIZE[1:0]位的值。ADC 数据寄存器只有低 16 位数据有效, 使用半字数据宽度。
- 13) DMA_MemoryDataSize: 存储器数据宽度, 可选字节(8 位)、半字(16 位)和字(32 位), 它设定 DMA_SxCR 寄存器的 MSIZE[1:0]位的值。保存 ADC 转换数据也要使用半字数据宽度, 这跟我们定义的数组是相对应的。
- 14) DMA_Mode: DMA 传输模式选择, 可选一次传输或者循环传输, 它设定 DMA_SxCR 寄存器的 CIRC 位的值。我们希望 ADC 采集是持续循环进行的, 所以使用循环传输模式。
- 15) DMA_Priority: 软件设置数据流的优先级, 有 4 个可选优先级分别为非常高、高、中和低, 它设定 DMA_SxCR 寄存器的 PL[1:0]位的值。DMA 优先级只有在多个 DMA 数据流同时使用时才有意义, 这里我们设置为非常高优先级就可以了。
- 16) DMA_FIFOMode: FIFO 模式使能, 如果设置为 DMA_FIFOMode_Enable 表示使能 FIFO 模式功能; 它设定 DMA_SxFCR 寄存器的 DMDIS 位。ADC 采集传输使用直接传输模式即可, 不需要使用 FIFO 模式。
- 17) DMA_FIFOTHreshold: FIFO 阈值选择, 可选 4 种状态分别为 FIFO 容量的 1/4、1/2、3/4 和满; 它设定 DMA_SxFCR 寄存器的 FTH[1:0]位; DMA_FIFOMode 设置为 DMA_FIFOMode_Disable, 那 DMA_FIFOTHreshold 值无效。ADC 采集传输不使用 FIFO 模式, 设置改值无效。
- 18) DMA_MemoryBurst: 存储器突发模式选择, 可选单次模式、4 节拍的增量突发模式、8 节拍的增量突发模式或 16 节拍的增量突发模式, 它设定 DMA_SxCR 寄存器的 MBURST[1:0]位的值。ADC 采集传输是直接模式, 要求使用单次模式。
- 19) DMA_PeripheralBurst: 外设突发模式选择, 可选单次模式、4 节拍的增量突发模式、8 节拍的增量突发模式或 16 节拍的增量突发模式, 它设定 DMA_SxCR 寄存器的 PBURST[1:0]位的值。ADC 采集传输是直接模式, 要求使用单次模式。

22.5 DMA 存储器到存储器模式实验

DMA 工作模式多样, 具体如何使用需要配合实际传输条件具体分析。接下来我们通过两个实验详细讲解 DMA 不同模式下的使用配置, 加深我们对 DMA 功能的理解。

DMA 运行高效, 使用方便, 在很多测试实验都会用到, 这里先详解存储器到存储器和存储器到外设这两种模式, 其他功能模式在其他章节会有很多使用到的情况, 也会有相关的分析。

存储器到存储器模式可以实现数据在两个内存的快速拷贝。我们先定义一个静态的源数据，然后使用 DMA 传输把源数据拷贝到目标地址上，最后对比源数据和目标地址的数据，看看是否传输准确。

22.5.1 硬件设计

DMA 存储器到存储器实验不需要其他硬件要求，只用到 RGB 彩色灯用于指示程序状态，关于 RGB 彩色灯电路可以参考 GPIO 章节。

22.5.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。这个实验代码比较简单，主要程序代码都在 main.c 文件中。

1. 编程要点

- 1) 使能 DMA 数据流时钟并复位初始化 DMA 数据流；
- 2) 配置 DMA 数据流参数；
- 3) 使能 DMA 数据流，进行传输；
- 4) 等待传输完成，并对源数据和目标地址数据进行比较。

2. 代码分析

DMA 宏定义及相关变量定义

代码清单 22-1 DMA 数据流和相关变量定义

```
1 /* 相关宏定义，使用存储器到存储器传输必须使用 DMA2 */
2 #define DMA_STREAM          DMA2_Stream0
3 #define DMA_CHANNEL         DMA_Channel_0
4 #define DMA_STREAM_CLOCK    RCC_AHB1Periph_DMA2
5 #define DMA_FLAG_TCIF       DMA_FLAG_TCIF0
6
7 #define BUFFER_SIZE         32
8 #define TIMEOUT_MAX         10000 /* Maximum timeout value */
9
10 /* 定义 aSRC_Const_Buffer 数组作为 DMA 传输数据源
11  const 关键字将 aSRC_Const_Buffer 数组变量定义为常量类型 */
12 const uint32_t aSRC_Const_Buffer[BUFFER_SIZE]= {
13     0x01020304,0x05060708,0x090A0B0C,0x0D0E0F10,
14     0x11121314,0x15161718,0x191A1B1C,0x1D1E1F20,
15     0x21222324,0x25262728,0x292A2B2C,0x2D2E2F30,
16     0x31323334,0x35363738,0x393A3B3C,0x3D3E3F40,
17     0x41424344,0x45464748,0x494A4B4C,0x4D4E4F50,
18     0x51525354,0x55565758,0x595A5B5C,0x5D5E5F60,
19     0x61626364,0x65666768,0x696A6B6C,0x6D6E6F70,
20     0x71727374,0x75767778,0x797A7B7C,0x7D7E7F80
```

```
21 };  
22 /* 定义 DMA 传输目标存储器 */  
23 uint32_t aDST_Buffer[BUFFER_SIZE];
```

使用宏定义设置外设配置方便程序修改和升级。

存储器到存储器传输必须使用 DMA2，但对数据流编号以及通道选择就没有硬性要求，可以自由选择。

aSRC_Const_Buffer[BUFFER_SIZE]是定义用来存放源数据的，并且使用了 const 关键字修饰，即常量类型，使得变量是存储在内部 flash 空间上。

DMA 数据流配置

代码清单 22-2 DMA 传输参数配置

```
1 static void DMA_Config(void)  
2 {  
3     DMA_InitTypeDef  DMA_InitStructure;  
4     __IO uint32_t      Timeout = TIMEOUT_MAX;  
5  
6     /* 使能 DMA 时钟 */  
7     RCC_AHB1PeriphClockCmd(DMA_STREAM_CLOCK, ENABLE);  
8  
9     /* 复位初始化 DMA 数据流 */  
10    DMA_DeInit(DMA_STREAM);  
11  
12    /* 确保 DMA 数据流复位完成 */  
13    while (DMA_GetCmdStatus(DMA_STREAM) != DISABLE) {  
14    }  
15  
16    /* DMA 数据流通道选择 */  
17    DMA_InitStructure.DMA_Channel = DMA_CHANNEL;  
18    /* 源数据地址 */  
19    DMA_InitStructure.DMA_PeripheralBaseAddr=(uint32_t)aSRC_Const_Buffer;  
20    /* 目标地址 */  
21    DMA_InitStructure.DMA_Memory0BaseAddr = (uint32_t)aDST_Buffer;  
22    /* 存储器到存储器模式 */  
23    DMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToMemory;  
24    /* 数据数目 */  
25    DMA_InitStructure.DMA_BufferSize = (uint32_t)BUFFER_SIZE;  
26    /* 使能自动递增功能 */  
27    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Enable;  
28    /* 使能自动递增功能 */  
29    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;  
30    /* 源数据是字大小(32 位) */  
31    DMA_InitStructure.DMA_PeripheralDataSize=DMA_PeripheralDataSize_Word;  
32    /* 目标数据也是字大小(32 位) */  
33    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;  
34    /* 一次传输模式，存储器到存储器模式不能使用循环传输 */  
35    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;  
36    /* DMA 数据流优先级为高 */  
37    DMA_InitStructure.DMA_Priority = DMA_Priority_High;  
38    /* 禁用 FIFO 模式 */  
39    DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;  
40    DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;  
41    /* 单次模式 */  
42    DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;  
43    /* 单次模式 */  
44    DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;  
45    /* 完成 DMA 数据流参数配置 */  
46    DMA_Init(DMA_STREAM, &DMA_InitStructure);  
47
```

```
48      /* 清除 DMA 数据流传输完成标志位 */
49      DMA_ClearFlag(DMA_STREAM, DMA_FLAG_TCIF);
50
51      /* 使能 DMA 数据流, 开始 DMA 数据传输 */
52      DMA_Cmd(DMA_STREAM, ENABLE);
53
54      /* 检测 DMA 数据流是否有效并带有超时检测功能 */
55      Timeout = TIMEOUT_MAX;
56      while ((DMA_GetCmdStatus(DMA_STREAM) != ENABLE) && (Timeout-- > 0)) {
57      }
58
59      /* 判断是否超时 */
60      if (Timeout == 0) {
61          /* 超时就让程序运行下面循环: RGB 彩色灯闪烁 */
62          while (1) {
63              LED_RED;
64              Delay(0xFFFFFF);
65              LED_RGBOFF;
66              Delay(0xFFFFFF);
67          }
68      }
69 }
```

使用 DMA_InitTypeDef 结构体定义一个 DMA 数据流初始化变量, 这个结构体内容我们之前已经有详细讲解。定义一个无符号 32 位整数变量 Timeout 用来计数超时。

调用 RCC_AHB1PeriphClockCmd 函数开启 DMA 数据流时钟, 使用 DMA 控制器之前必须开启对应的时钟。

DMA_DeInit 函数见数据流复位到缺省配置状态。

使用 DMA_GetCmdStatus 函数获取当前 DMA 数据流状态, 该函数接收一个 DMA 数据流的参数, 返回当前数据流状态, 复位 DMA 数据流之前需要调用该函数来确保 DMA 数据流复位完成。

存储器到存储器模式通道选择没有具体规定, 源地址和目标地址使用之前定义的数组首地址, 只能使用一次传输模式不能循环传输, 最后我调用 DMA_Init 函数完成 DMA 数据流的初始化配置。

DMA_ClearFlag 函数用于清除 DMA 数据流标志位, 代码用到传输完成标志位, 使用之前清除标志位以免产生不必要干扰。DMA_ClearFlag 函数需要两个形参, 一个是 DMA 数据流, 一个是事件标志位, 可选有数据流传输完成标志位、半传输标志位、FIFO 错误标志位、传输错误标志位以及直接模式错误标志位。

DMA_Cmd 函数用于启动或者停止 DMA 数据流传输, 它接收连个参数, 第一个是 DMA 数据流, 另外一个开启 ENABLE 或者停止 DISABLE。

开启 DMA 传输后需要使用 DMA_GetCmdStatus 函数获取 DMA 数据流状态, 确保 DMA 数据流配置有效, 为防止程序卡死, 添加了超时检测功能。

如果 DMA 配置超时错误闪烁 RGB 彩灯提示。

存储器数据对比

代码清单 22-3 源数据与目标地址数据对比

```
1 uint8_t Buffercmp(const uint32_t* pBuffer,
2                   uint32_t* pBuffer1, uint16_t BufferLength)
3 {
```



```
4    /* 数据长度递减 */
5    while (BufferLength--) {
6        /* 判断两个数据源是否对应相等 */
7        if (*pBuffer != *pBuffer1) {
8            /* 对应数据源不相等马上退出函数，并返回 0 */
9            return 0;
10       }
11       /* 递增两个数据源的地址指针 */
12       pBuffer++;
13       pBuffer1++;
14   }
15   /* 完成判断并且对应数据相对 */
16   return 1;
17 }
```

判断指定长度的两个数据源是否完全相等，如果完全相等返回 1；只要其中一对数据不相等返回 0。它需要三个形参，前两个是两个数据源的地址，第三个是要比较数据长度。

主函数

代码清单 22-4 存储器到存储器模式主函数

```
1 int main(void)
2 {
3     /* 定义存放比较结果变量 */
4     uint8_t TransferStatus;
5
6     /* LED 端口初始化 */
7     LED_GPIO_Config();
8
9     /* 设置 RGB 彩色灯为紫色 */
10    LED_PURPLE;
11
12    /* 简单延时函数 */
13    Delay(0xFFFFF);
14
15    /* DMA 传输配置 */
16    DMA_Config();
17
18    /* 等待 DMA 传输完成 */
19    while (DMA_GetFlagStatus(DMA_STREAM, DMA_FLAG_TCIF) == DISABLE) {
20
21    }
22
23    /* 比较源数据与传输后数据 */
24    TransferStatus = Buffercmp(aSRC_Const_Buffer, aDST_Buffer, BUFFER_SIZE);
25
26    /* 判断源数据与传输后数据比较结果 */
27    if (TransferStatus == 0) {
28        /* 源数据与传输后数据不相等时 RGB 彩色灯显示红色 */
29        LED_RED;
30    } else {
31        /* 源数据与传输后数据相等时 RGB 彩色灯显示蓝色 */
32        LED_BLUE;
33    }
34
35    while (1) {
36    }
37 }
```

首先定义一个变量用来保存存储器数据比较结果。

RGB 彩色灯用来指示程序进程，使用之前需要初始化它，LED_GPIO_Config 定义在 bsp_led.c 文件中。开始设置 RGB 彩色灯为紫色，LED_PURPLE 是定义在 bsp_led.h 文件的一个宏定义。

Delay 函数只是一个简单的延时函数。

调用 DMA_Config 函数完成 DMA 数据流配置并启动 DMA 数据传输。

DMA_GetFlagStatus 函数获取 DMA 数据流事件标志位的当前状态，这里获取 DMA 数据传输完成这个标志位，使用循环持续等待直到该标志位被置位，即 DMA 传输完成这个事件发生，然后退出循环，运行之后程序。

确定 DMA 传输完成之后就可以调用 Buffercmp 函数比较源数据与 DMA 传输后目标地址的数据是否一一对应。TransferStatus 保存比较结果，如果为 1 表示两个数据源一一对应相等说明 DMA 传输成功；相反，如果为 0 表示两个数据源数据存在不等情况，说明 DMA 传输出错。

如果 DMA 传输成功设置 RGB 彩色灯为蓝色，如果 DMA 传输出错设置 RGB 彩色灯为红色。

22.5.3 下载验证

确保开发板供电正常，编译程序并下载。观察 RGB 彩色灯变化情况。正常情况下 RGB 彩色灯先为紫色，然后变成蓝色。如果 DMA 传输出错才会为红色。

22.6 DMA 存储器到外设模式实验

DMA 存储器到外设传输模式非常方便把存储器数据传输到外设数据寄存器中，这在 STM32 芯片向其他目标主机，比如电脑、另外一块开发板或者功能芯片，发送数据是非常有用的。RS-232 串口通信是我们常用开发板与 PC 端通信的方法。我们可以使用 DMA 传输把指定的存储器数据转移到 USART 数据寄存器内，并发送至 PC 端，在串口调试助手显示。

22.6.1 硬件设计

存储器到外设模式使用到 USART1 功能，具体电路设置参考 USART 章节，无需其他硬件设计。

22.6.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们编写两个串口驱动文件 bsp_usart_dma.c 和 bsp_usart_dma.h，有关串口和 DMA 的宏定义以及驱动函数都在里边。

1. 编程要点

- 1) 配置 USART 通信功能；

- 2) 设置 DMA 为存储器到外设模式，设置数据流通道，指定 USART 数据寄存器为目标地址，循环发送模式；
- 3) 使能 DMA 数据流；
- 4) 使能 USART 的 DMA 发送请求；
- 5) DMA 传输同时 CPU 可以运行其他任务。

2. 代码分析

USART 和 DMA 宏定义

代码清单 22-5 USART 和 DMA 相关宏定义

```
1 //USART
2 #define DEBUG_USART                USART1
3 #define DEBUG_USART_CLK            RCC_APB2Periph_USART1
4 #define DEBUG_USART_RX_GPIO_PORT  GPIOA
5 #define DEBUG_USART_RX_GPIO_CLK   RCC_AHB1Periph_GPIOA
6 #define DEBUG_USART_RX_PIN         GPIO_Pin_10
7 #define DEBUG_USART_RX_AF          GPIO_AF_USART1
8 #define DEBUG_USART_RX_SOURCE      GPIO_PinSource10
9
10 #define DEBUG_USART_TX_GPIO_PORT   GPIOA
11 #define DEBUG_USART_TX_GPIO_CLK    RCC_AHB1Periph_GPIOA
12 #define DEBUG_USART_TX_PIN         GPIO_Pin_9
13 #define DEBUG_USART_TX_AF          GPIO_AF_USART1
14 #define DEBUG_USART_TX_SOURCE      GPIO_PinSource9
15
16 #define DEBUG_USART_BAUDRATE        115200
17
18 //DMA
19 #define DEBUG_USART_DR_BASE          (USART1_BASE+0x04)
20 #define SENDBUFF_SIZE                5000 //一次发送的数据量
21 #define DEBUG_USART_DMA_CLK          RCC_AHB1Periph_DMA2
22 #define DEBUG_USART_DMA_CHANNEL      DMA_Channel_4
23 #define DEBUG_USART_DMA_STREAM      DMA2_Stream7
```

使用宏定义设置外设配置方便程序修改和升级。

USART 部分设置与 USART 章节内容相同，可以参考 USART 章节内容理解。

查阅表 22-2 可知 USART1 对应 DMA2 的数据流 7 通道 4。

串口 DMA 传输配置

代码清单 22-6 USART1 发送请求 DMA 设置

```
1 void USART_DMA_Config(void)
2 {
3     DMA_InitTypeDef DMA_InitStructure;
4
5     /*开启 DMA 时钟*/
6     RCC_AHB1PeriphClockCmd(DEBUG_USART_DMA_CLK, ENABLE);
7
8     /* 复位初始化 DMA 数据流 */
9     DMA_DeInit(DEBUG_USART_DMA_STREAM);
10 }
```

```
11  /* 确保 DMA 数据流复位完成 */
12  while (DMA_GetCmdStatus(DEBUG_USART_DMA_STREAM) != DISABLE) {
13  }
14
15  /*usart1 tx 对应 dma2, 通道 4, 数据流 7*/
16  DMA_InitStructure.DMA_Channel = DEBUG_USART_DMA_CHANNEL;
17  /*设置 DMA 源: 串口数据寄存器地址*/
18  DMA_InitStructure.DMA_PeripheralBaseAddr = DEBUG_USART_DR_BASE;
19  /*内存地址 (要传输的变量的指针)*/
20  DMA_InitStructure.DMA_Memory0BaseAddr = (u32)SendBuff;
21  /*方向: 从内存到外设*/
22  DMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToPeripheral;
23  /*传输大小 DMA_BufferSize=SENDERBUFF_SIZE*/
24  DMA_InitStructure.DMA_BufferSize = SENDERBUFF_SIZE;
25  /*外设地址不增*/
26  DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
27  /*内存地址自增*/
28  DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
29  /*外设数据单位*/
30  DMA_InitStructure.DMA_PeripheralDataSize=DMA_PeripheralDataSize_Byte;
31  /*内存数据单位 8bit*/
32  DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Byte;
33  /*DMA 模式: 不断循环*/
34  DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
35  /*优先级: 中*/
36  DMA_InitStructure.DMA_Priority = DMA_Priority_Medium;
37  /*禁用 FIFO*/
38  DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
39  DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
40  /*存储器突发传输 16 个节拍*/
41  DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single;
42  /*外设突发传输 1 个节拍*/
43  DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
44  /*配置 DMA2 的数据流 7*/
45  DMA_Init(DEBUG_USART_DMA_STREAM, &DMA_InitStructure);
46
47  /*使能 DMA*/
48  DMA_Cmd(DEBUG_USART_DMA_STREAM, ENABLE);
49
50  /* 等待 DMA 数据流有效*/
51  while (DMA_GetCmdStatus(DEBUG_USART_DMA_STREAM) != ENABLE) {
52  }
53 }
```

使用 DMA_InitTypeDef 结构体定义一个 DMA 数据流初始化变量, 这个结构体内容我们之前已经有详细讲解。

调用 RCC_AHB1PeriphClockCmd 函数开启 DMA 数据流时钟, 使用 DMA 控制器之前必须开启对应的时钟。

DMA_DeInit 函数见数据流复位到缺省配置状态。

使用 DMA_GetCmdStatus 函数获取当前 DMA 数据流状态, 该函数接收一个 DMA 数据流的参数, 返回当前数据流状态, 复位 DMA 数据流之前需要调用该函数来确保 DMA 数据流复位完成。

USART 有固定的 DMA 通道, USART 数据寄存器地址也是固定的, 外设地址不可以使用自动递增, 源数据使用我们自定义的数组空间, 存储器地址使用自动递增, 采用循环发送模式, 最后我调用 DMA_Init 函数完成 DMA 数据流的初始化配置。

DMA_Cmd 函数用于启动或者停止 DMA 数据流传输，它接收连个参数，第一个是 DMA 数据流，另外一个开启 ENABLE 或者停止 DISABLE。

开启 DMA 传输后需要使用 DMA_GetCmdStatus 函数获取 DMA 数据流状态，确保 DMA 数据流配置有效。

主函数

代码清单 22-7 存储器到外设模式主函数

```
1 int main(void)
2 {
3     uint16_t i;
4     /* 初始化 USART */
5     Debug_USART_Config();
6
7     /* 配置使用 DMA 模式 */
8     USART_DMA_Config();
9
10    /* 配置 RGB 彩色灯 */
11    LED_GPIO_Config();
12
13    printf("\r\n USART1 DMA TX 测试 \r\n");
14
15    /*填充将要发送的数据*/
16    for (i=0; i<SENDBUFF_SIZE; i++) {
17        SendBuff[i] = 'A';
18    }
19
20
21    /* USART1 向 DMA 发出 TX 请求 */
22    USART_DMACmd(DEBUG_USART, USART_DMAREq_Tx, ENABLE);
23
24    /* 此时 CPU 是空闲的，可以干其他的事情 */
25    //例如同时控制 LED
26    while (1) {
27        LED1_TOGGLE
28        Delay(0xFFFFF);
29    }
30 }
```

Debug_USART_Config 函数定义在 bsp_usart_dma.c 中，它完成 USART 初始化配置，包括 GPIO 初始化，USART 通信参数设置等等，具体可参考 USART 章节讲解。

USART_DMA_Config 函数也是定义在 bsp_usart_dma.c 中，之前我们已经详细分析了。

LED_GPIO_Config 函数定义在 bsp_led.c 中，它完成 RGB 彩色灯初始化配置，具体可参考 GPIO 章节讲解。

使用 for 循环填充源数据，SendBuff[SENDBUFF_SIZE]是定义在 bsp_usart_dma.c 中的一个全局无符号 8 位整数数组，是 DMA 传输的源数据，在 USART_DMA_Config 函数中已经被设置为存储器地址。

USART_DMACmd 函数用于控制 USART 的 DMA 传输启动和关闭。它接收三个参数，第一个参数用于设置 DMA 数据流，第二个参数设置 DMA 请求，有 USART 发送请求 USART_DMAREq_Tx 和接收请求 USART_DMAREq_Rx 可选，第三个参数用于设置启动请求 ENABLE 或者关闭请求 DISABLE。运行该函数后 USART 的 DMA 发送传输就开始了，根据配置它会通过 USART 循环发送数据。

DMA 传输过程是不占用 CPU 资源的，可以一边传输一次运行其他任务。

22.6.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。程序运行后在串口调试助手可接收到大量的数据，同时开发板上 RGB 彩色灯不断闪烁。

这里要注意为演示 DMA 持续运行并且 CPU 还能处理其它事情，持续使用 DMA 发送数据，量非常大，长时间运行可能会导致电脑端串口调试助手会卡死，鼠标乱飞的情况，所以在测试时最好把串口调试助手的自动清除接收区数据功能勾选上或把 DMA 配置中的循环模式改为单次模式。

